

Retrieval and Retrieval-Augmented Generation

PGS.TS. Nguyễn Phương Thái

TS. Trần Hồng Việt

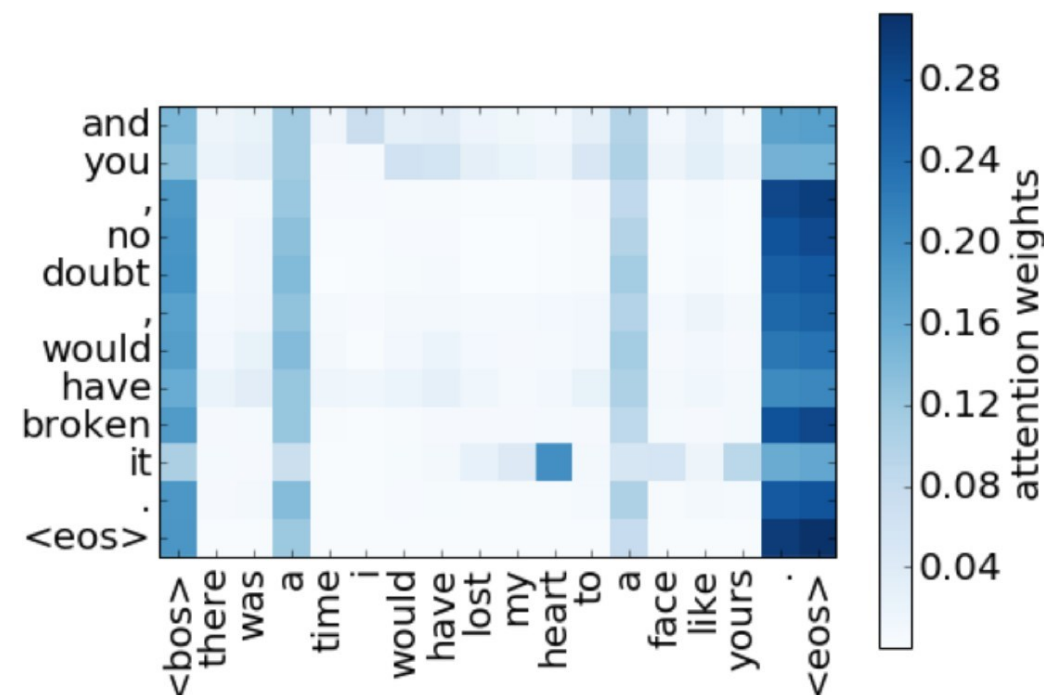
NLP Laboratory, Institute of Artificial Intelligence

Adapted from slides of <https://phontron.com/class/anlp2022/>

Long-context Transformers

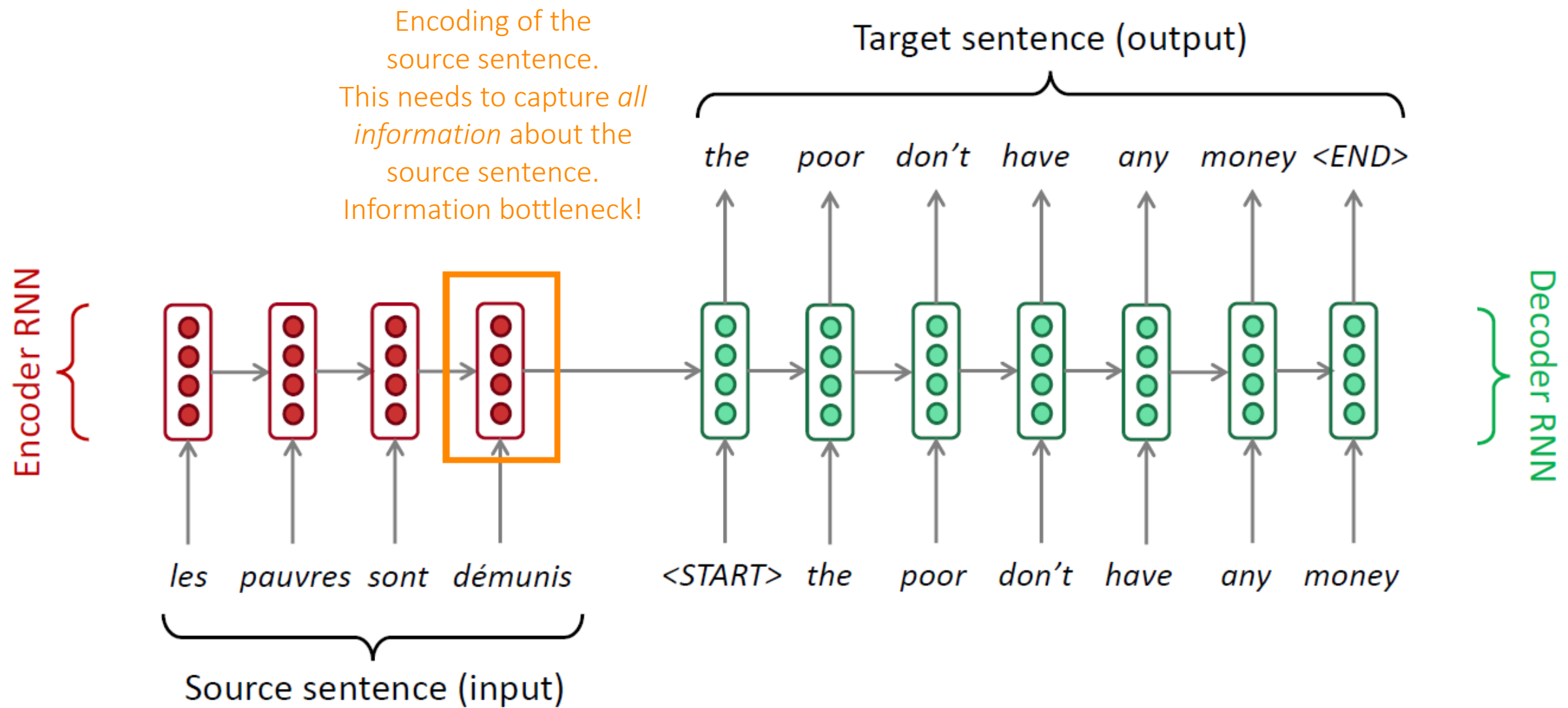
Training Transformers Over Longer Sequences

- Simply attend to all previous words in the document (e.g. Voita et al. 2018)
- + Can relatively simply use document-level context
- + Can learn interesting phenomena (e.g. co-reference)



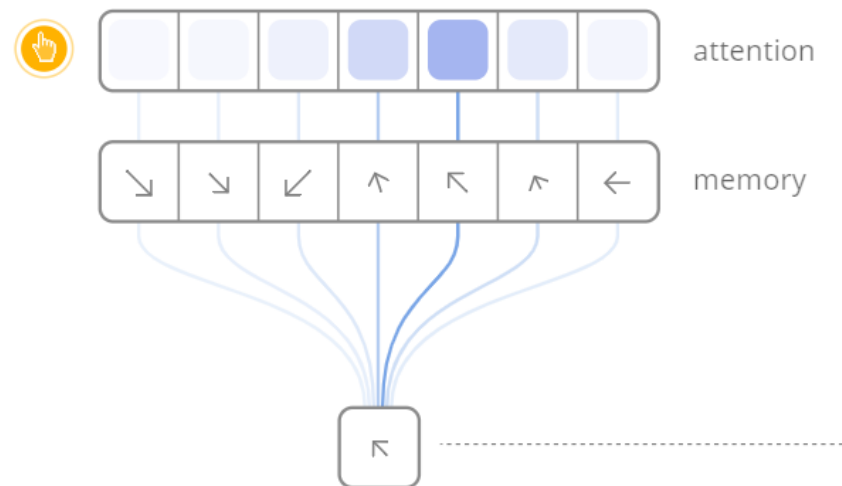
- - Computation is quadratic in sequence length!

Sequence-to-sequence: the bottleneck problem



Attention is a *general* Deep Learning technique

- More general definition of attention
 - Given a set of vector *values*, and a vector *query*, **attention** is a technique to compute a **weighted sum of the values**, dependent on the query.
 - We sometimes say that the query *attends to* the values
 - For example, in the seq2seq + attention model, each decoder hidden state *attends to* the encoder hidden states

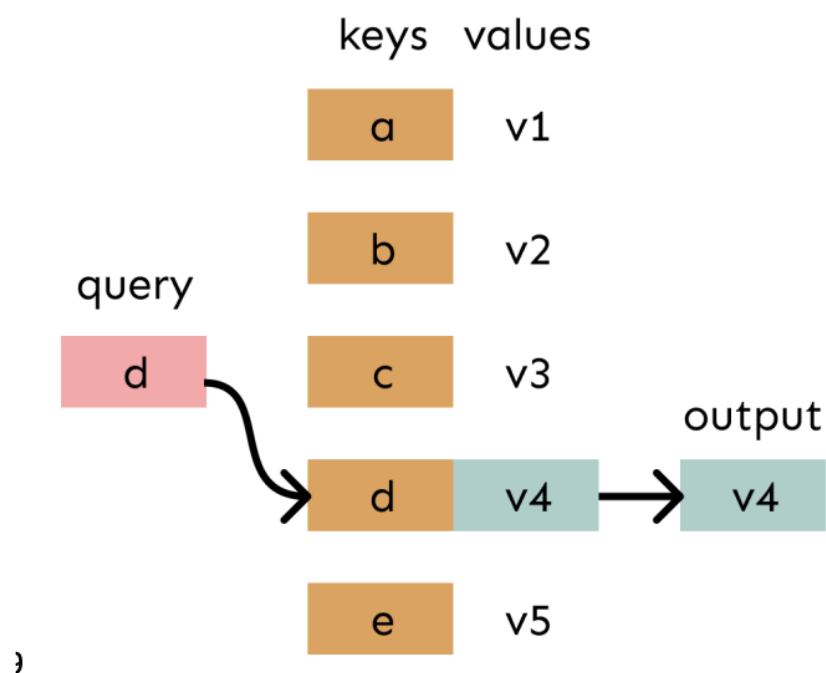


<https://distill.pub/2016/augmented-rnns/>

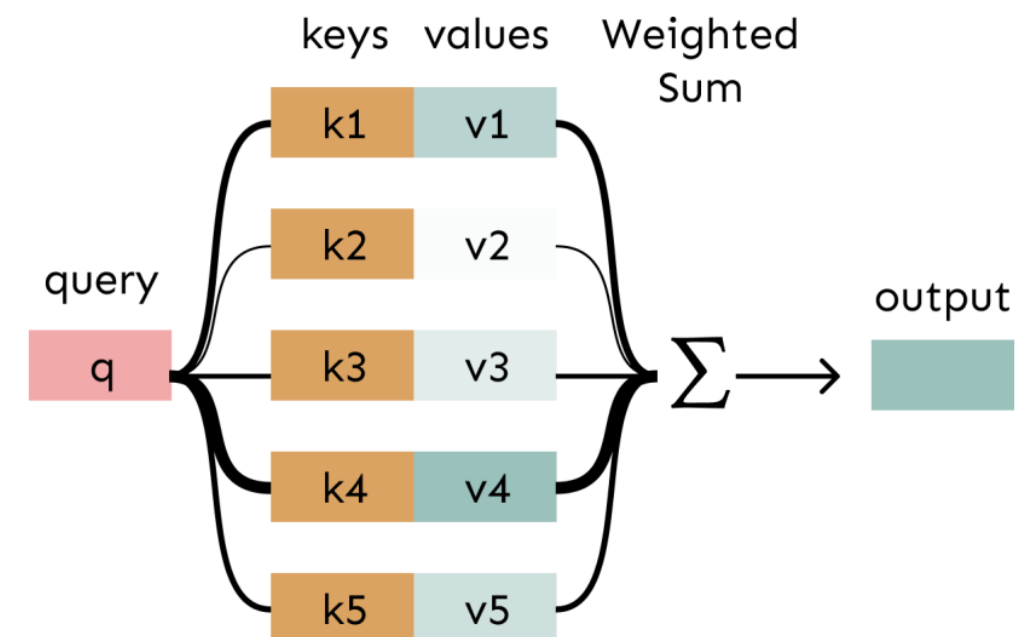
Attention as a soft, averaging lookup table

- We can think of **attention** as performing fuzzy lookup a in **key-value store**

Lookup table: a table of keys that map to values. The query matches one of the keys, returning its value.



Attention: The query matches to all keys softly to a weight between 0 and 1. The keys' values are multiplied by the weights and summed.



Problems with RNNs = Motivation for Transformers

- Recurrent models typically factor computation along the symbol positions of the input and output sequences
 - Sequential computation prevents parallelization
 - Critical at longer sequence lengths, as memory constraints limit batching across examples
- Despite advanced RNNs like LSTMs, RNNs still need attention mechanism to deal with long range dependencies – path length for codependent computation between states grows with sequence
- But if attention gives us access to any state... maybe we don't need the RNN?

Transformers (2017)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

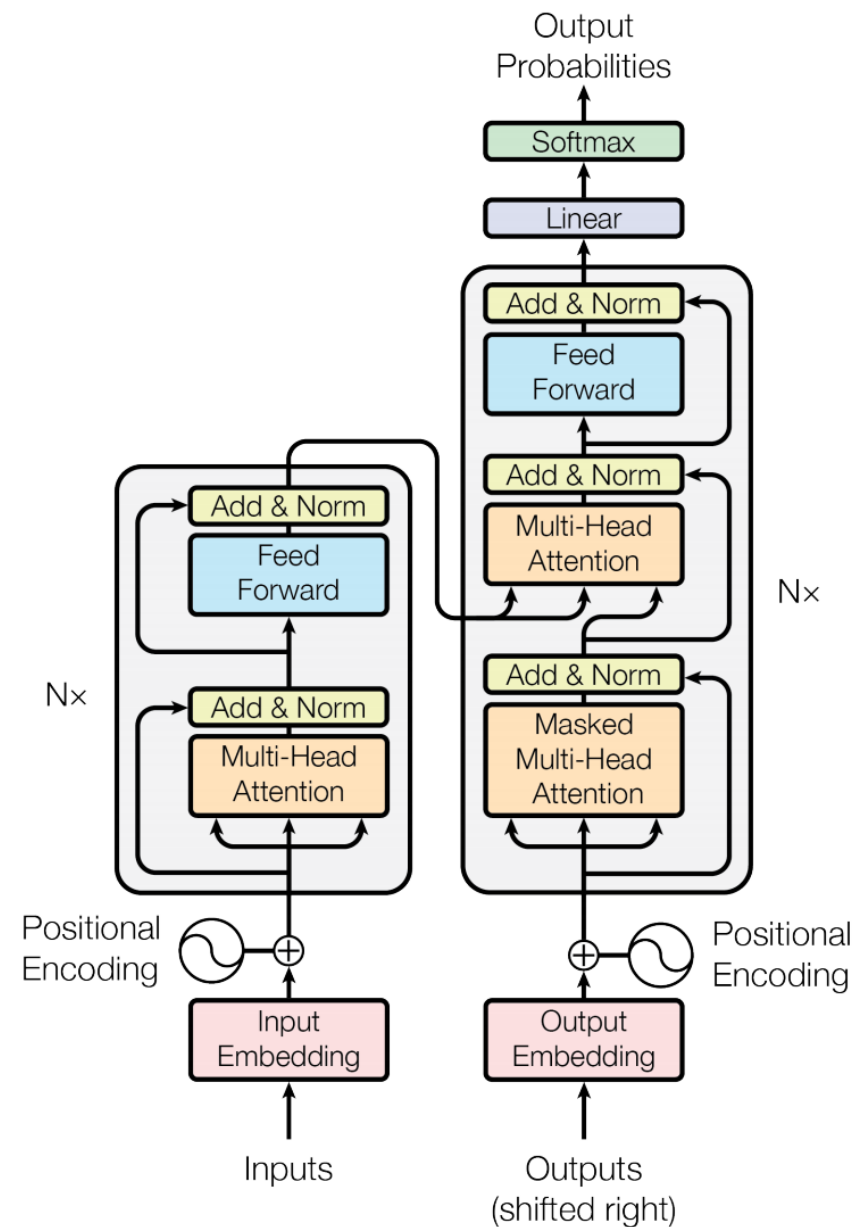
Illia Polosukhin* ‡
illia.polosukhin@gmail.com



(Vaswani et al., 2017)

Transformer Overview

- Sequence-to-sequence
- Encoder-Decoder
- Task: machine translation with parallel corpus
- Predict each translated word
- Final cost/error function is standard cross-entropy error on top of a softmax classifier



Self-Attention: keys, queries, values from the same sequence

Let $\mathbf{w}_{1:n}$ be a sequence of words in vocabulary V , like *Zuko made his uncle tea*.

For each $\mathbf{w}_i$, let $\mathbf{x}_i = E\mathbf{w}_i$, where $E \in \mathbb{R}^{d \times |V|}$ is an embedding matrix.

1. Transform each word embedding with weight matrices Q, K, V , each in $\mathbb{R}^{d \times d}$

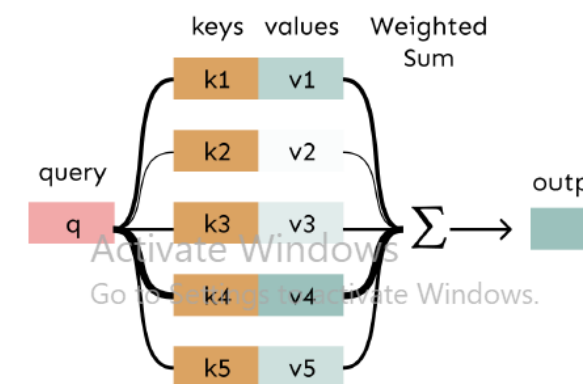
$$\mathbf{q}_i = Q\mathbf{x}_i \text{ (queries)} \quad \mathbf{k}_i = K\mathbf{x}_i \text{ (keys)} \quad \mathbf{v}_i = V\mathbf{x}_i \text{ (values)}$$

2. Compute pairwise similarities between keys and queries; normalize with softmax

$$\mathbf{e}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j \quad \alpha_{ij} = \frac{\exp(\mathbf{e}_{ij})}{\sum_{j'} \exp(\mathbf{e}_{ij'})}$$

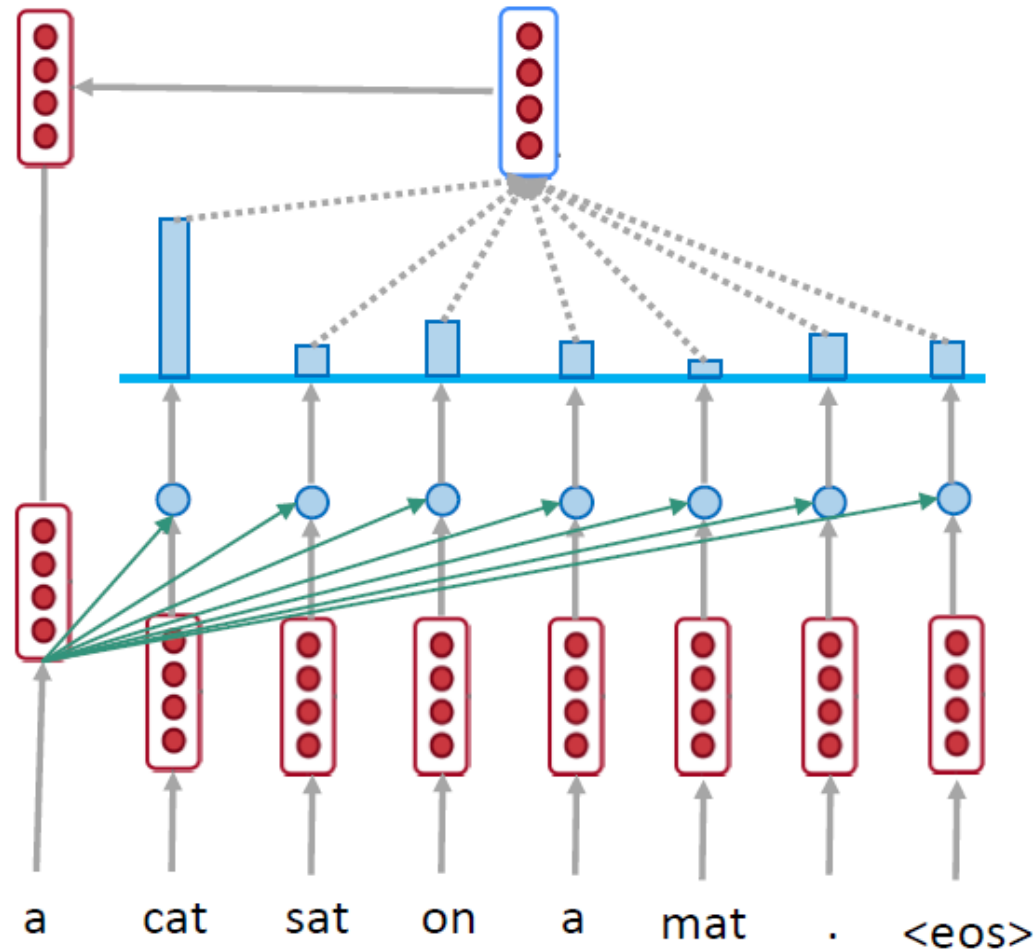
3. Compute output for each word as weighted sum of values

$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$



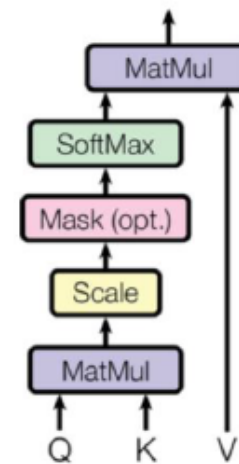
Self-attention: A Running Example

update
representation
for the word "a"

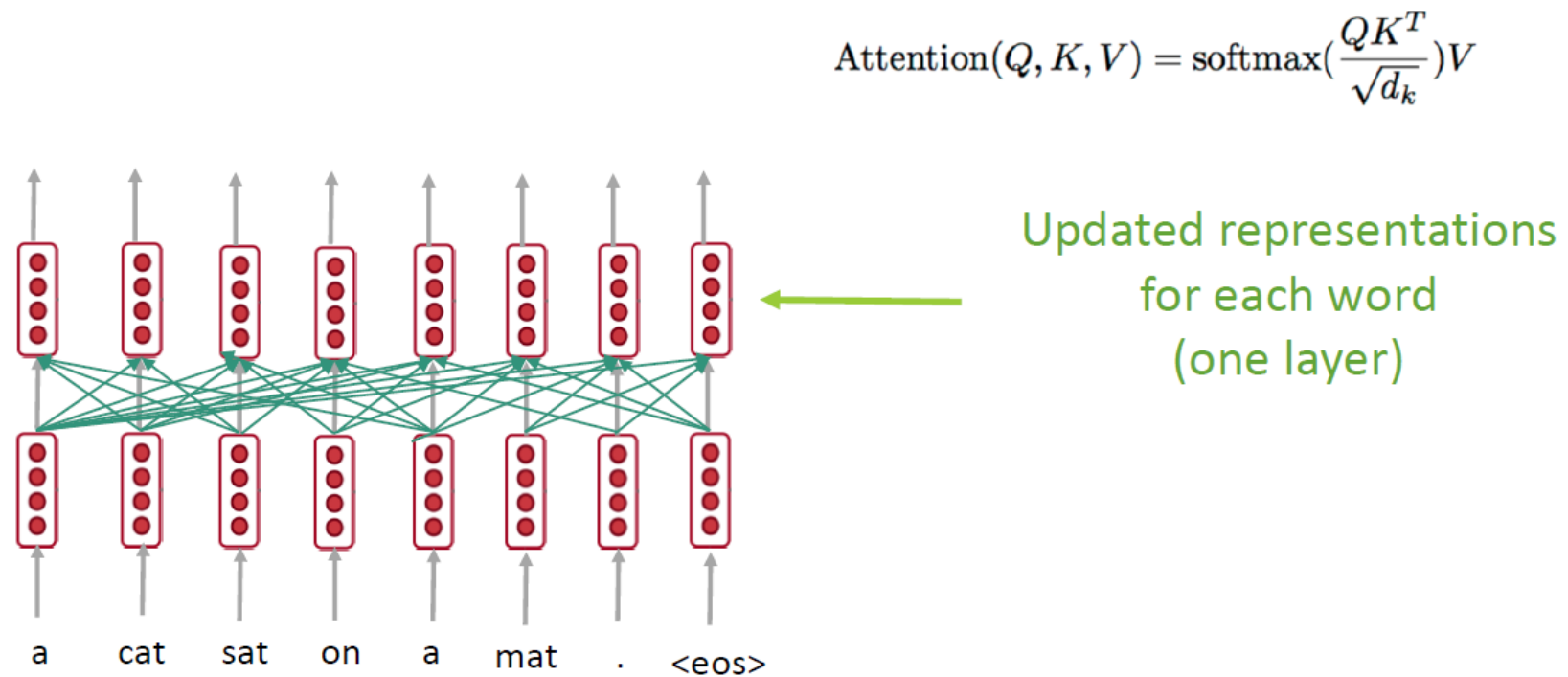


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



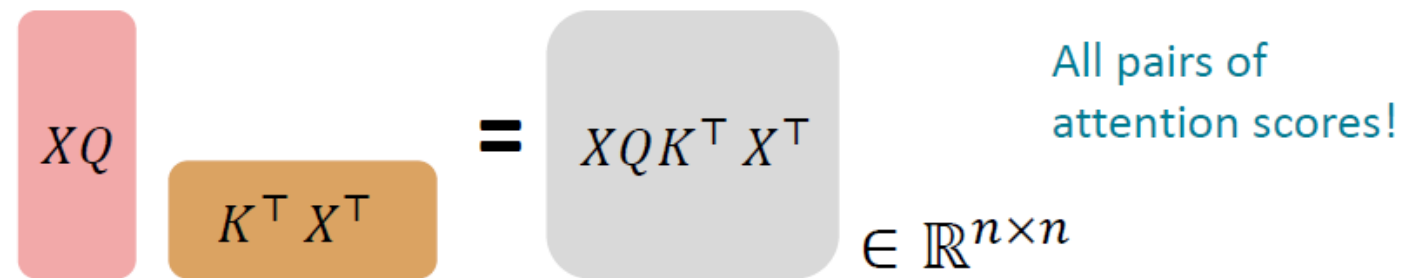
Self-attention: A Running Example



Sequence-stacked form of Attention

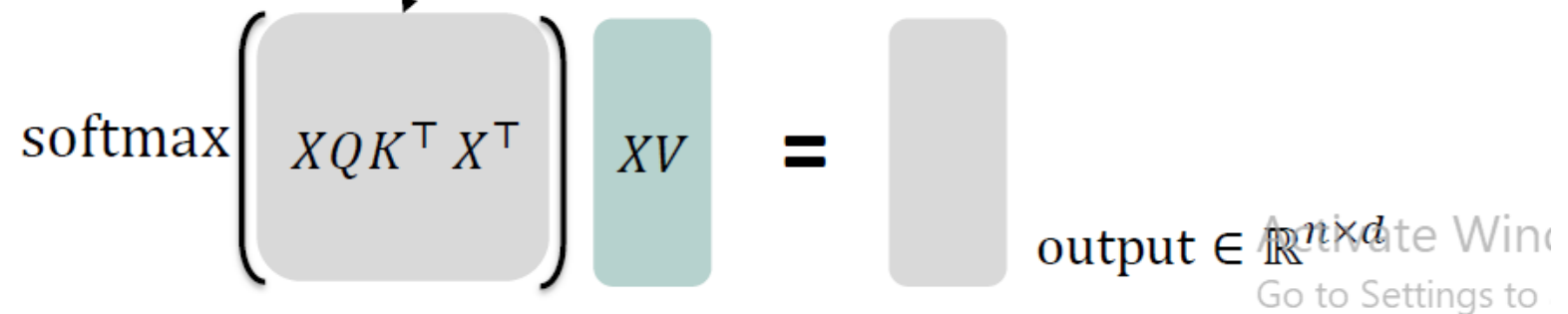
- Let's look at how key-query-value attention is computed, in matrices.
 - Let $X = [x_1; \dots; x_n] \in \mathbb{R}^{n \times d}$ be the concatenation of input vectors.
 - First, note that $XK \in \mathbb{R}^{n \times d}$, $XQ \in \mathbb{R}^{n \times d}$, $XV \in \mathbb{R}^{n \times d}$.
 - The output is defined as $\text{output} = \text{softmax}(XQ(XK)^T)XV \in \mathbb{R}^{n \times d}$.

First, take the query-key dot products in one matrix multiplication: $XQ(XK)^T$



A diagram illustrating the first step of the attention mechanism. On the left, a pink vertical rectangle labeled XQ is multiplied by an orange horizontal rectangle labeled $K^T X^T$. An equals sign follows, leading to a gray rounded rectangle labeled $XQK^T X^T$. To the right of this rectangle is the text $\in \mathbb{R}^{n \times n}$. Further to the right, the text "All pairs of attention scores!" is written in blue.

Next, softmax, and compute the weighted average with another matrix multiplication.



A diagram illustrating the second step of the attention mechanism. On the left, the text "softmax" is followed by a large left square bracket. Inside the bracket is a gray rounded rectangle labeled $XQK^T X^T$. To the right of the bracket is a teal vertical rectangle labeled XV . An equals sign follows, leading to a gray vertical rectangle. To the right of this rectangle is the text "output $\in \mathbb{R}^{n \times d}$ ".

The Transformer Encoder: Key-Query-Value Attention

- We saw that self-attention is when keys, queries, and values come from the same source. The Transformer does this in a particular way:
 - Let $x_1, \dots, x_n$ be input vectors to the Transformer encoder; $x_i \in \mathbb{R}^d$
- Then keys, queries, values are:
 - $k_i = Kx_i$, where $K \in \mathbb{R}^{d \times d}$ is the key matrix.
 - $q_i = Qx_i$, where $Q \in \mathbb{R}^{d \times d}$ is the query matrix.
 - $v_i = Vx_i$, where $V \in \mathbb{R}^{d \times d}$ is the value matrix.
- These matrices allow *different aspects* of the x vectors to be used/emphasized in each of the three roles.

Encoder Input

- Actual word representations are byte-pair encodings (subword)
 - Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. ACL 2016.
- Solution: Add “positional encoding” to the input embeddings: $\mathbf{p}_i$ for $i \in \mathbb{R}^d$ $i = 1, 2, \dots, n$
- Added is a positional encoding so same words at different locations have different overall representations (sine and cosine functions of different frequencies):

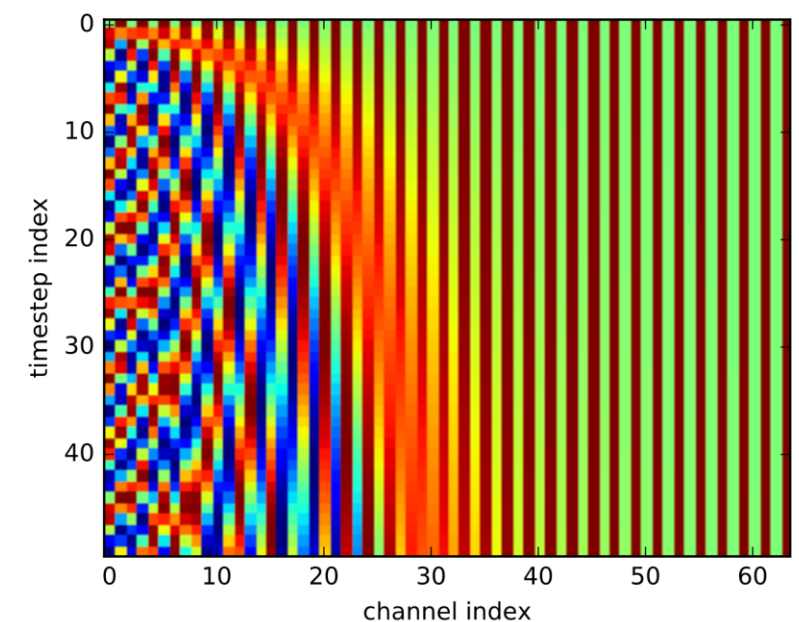
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{p}_i$$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

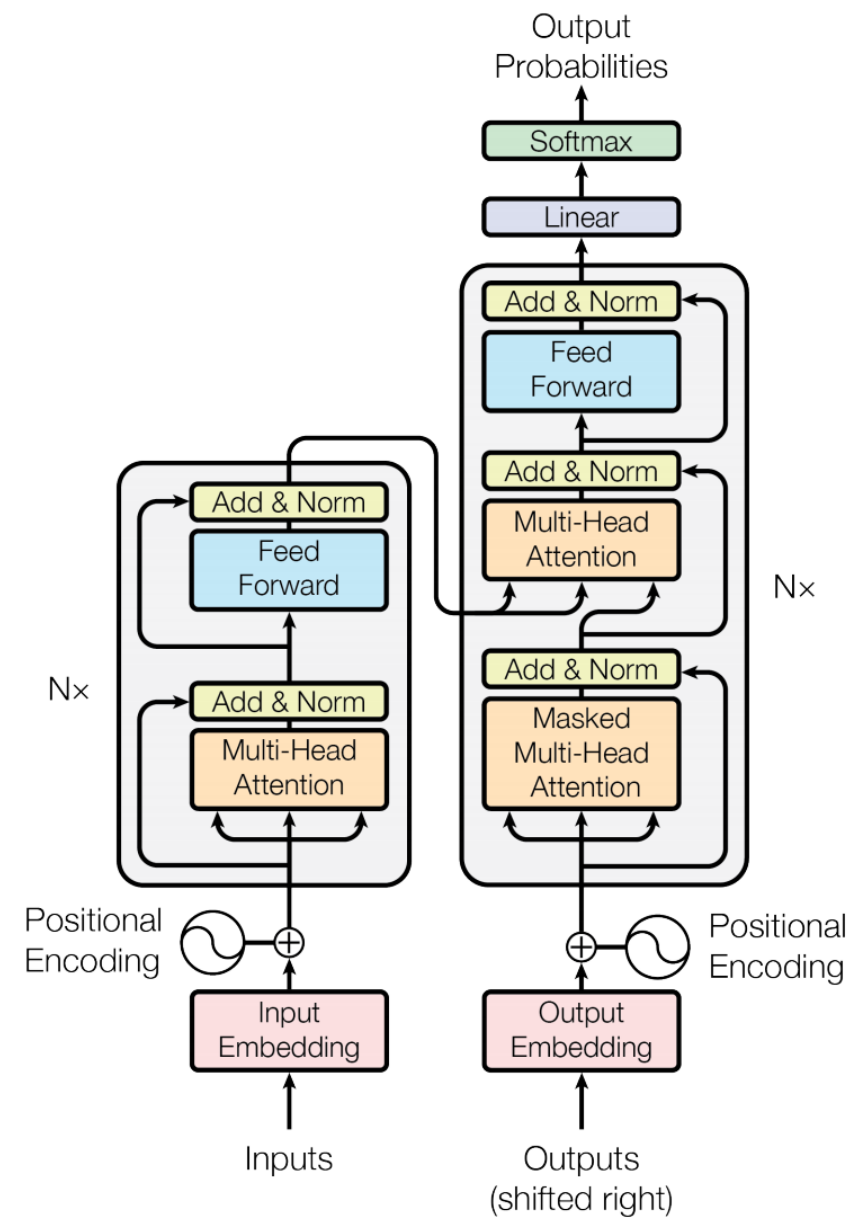
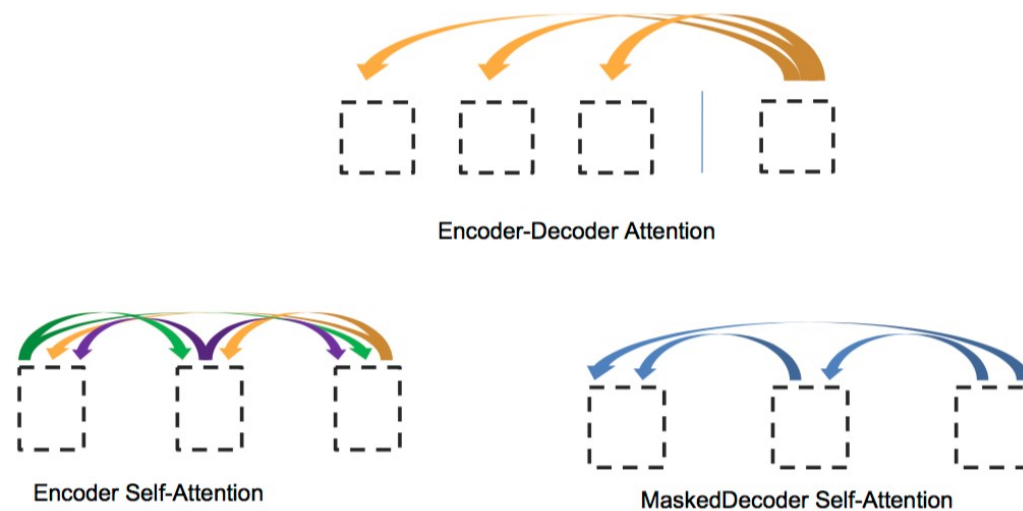
pos is the position of a word

i is the dimension index



Transformer Decoder

- 2 sublayer changes in decoder
- Masked decoder self-attention
 - Only depends on previous words
- Encoder-Decoder Attention
 - Queries come from previous decoder layer and keys and values come from output of encoder



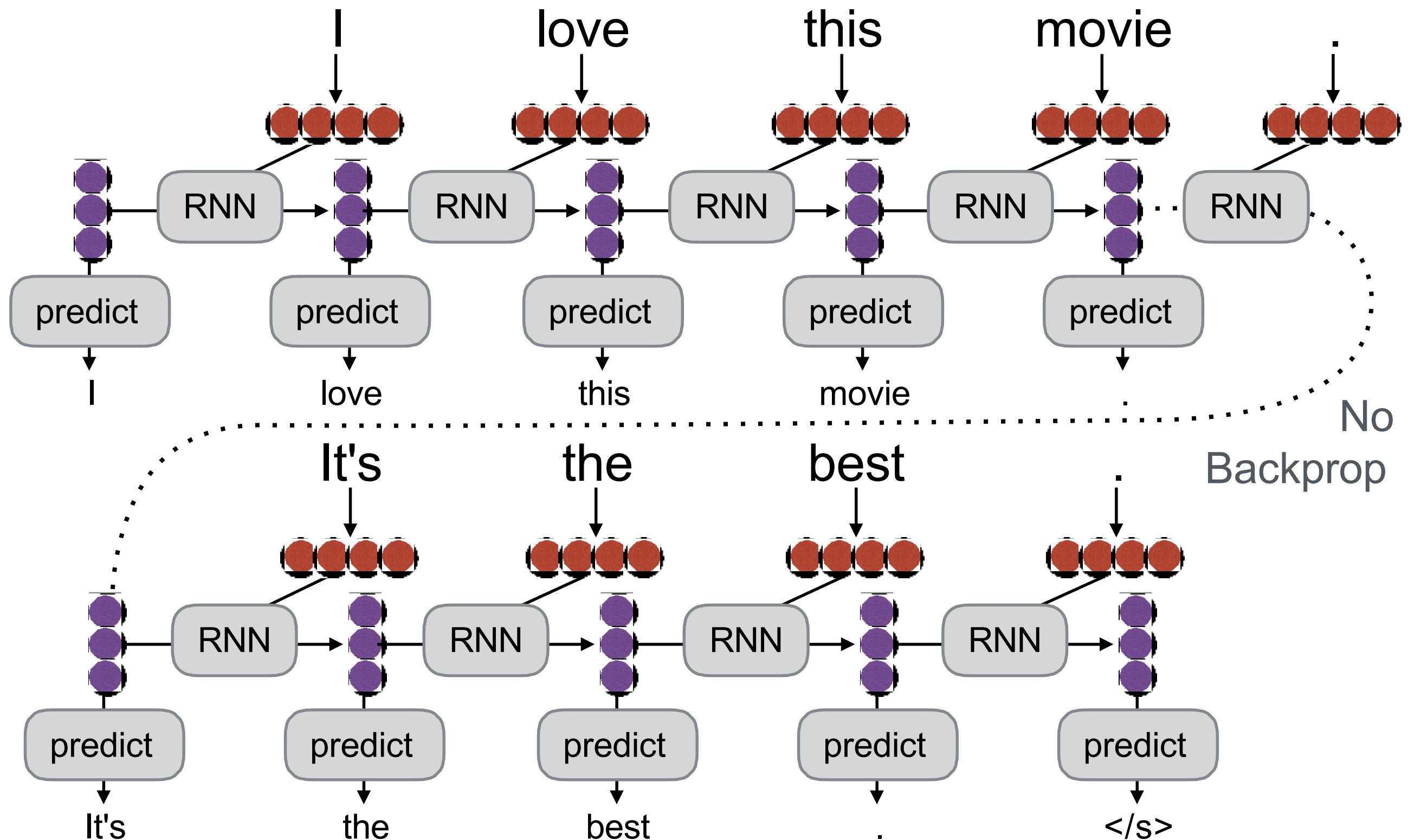
Practice

- Understanding Transformer deeply: <https://jalammar.github.io/illustrated-transformer/>
- Steps for implement Transformer:
<http://nlp.seas.harvard.edu/annotated-transformer/>
- ViNMT: Neural Machine Translation Toolkit
 - https://github.com/KCDichDaNgu/KC4.0_MultilingualNMT

Application of Transformer

- Transformers is the state of the art for any application of NLPs.
- Pretraining models for Transformer: Encoder (Bert), Decoder (GPT3, ChatGPT), Encoder-Decoder (T5, Bart)
- Now, Transformer is success for **Foundation Models**.

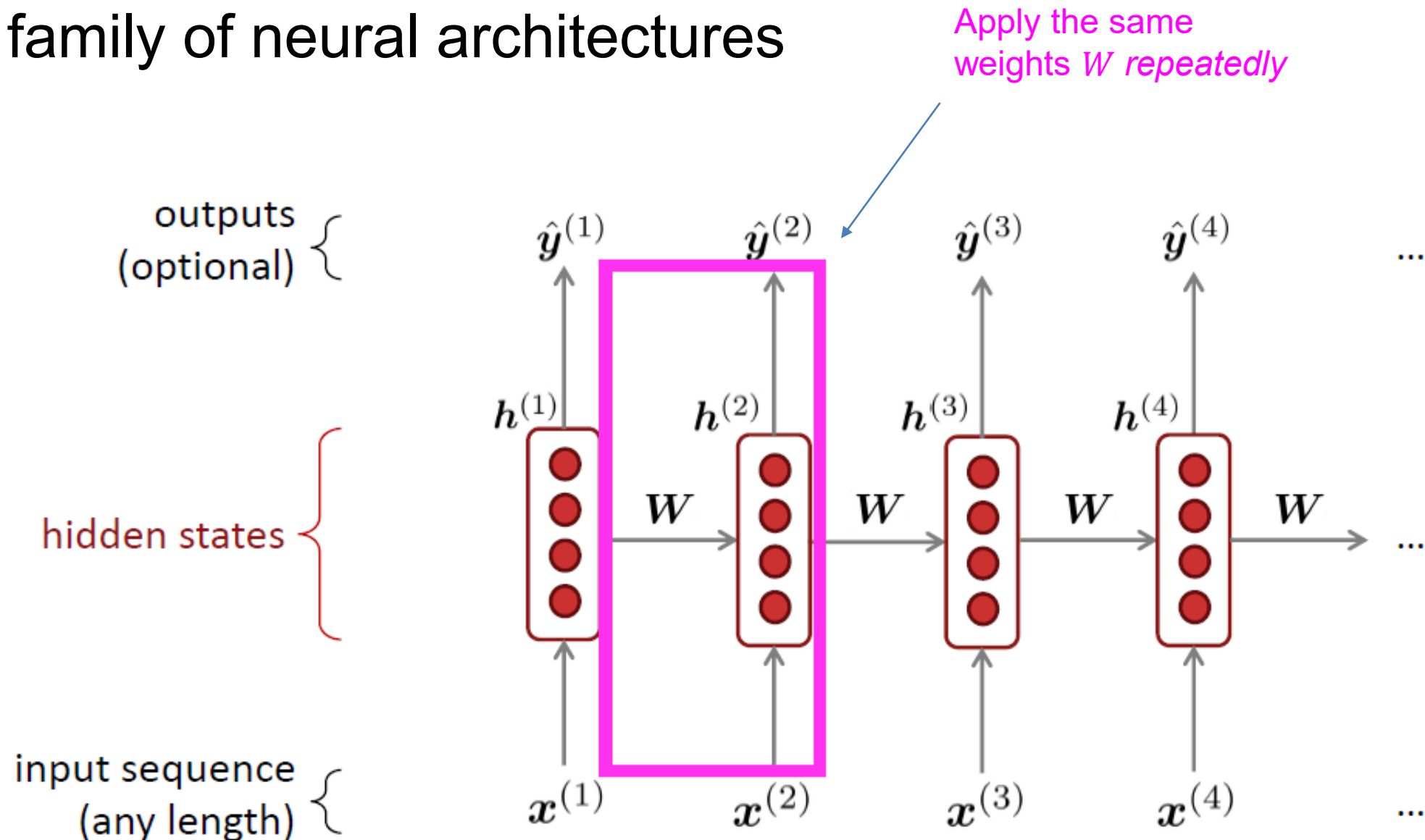
In RNNs: Pass State + Truncated Backprop



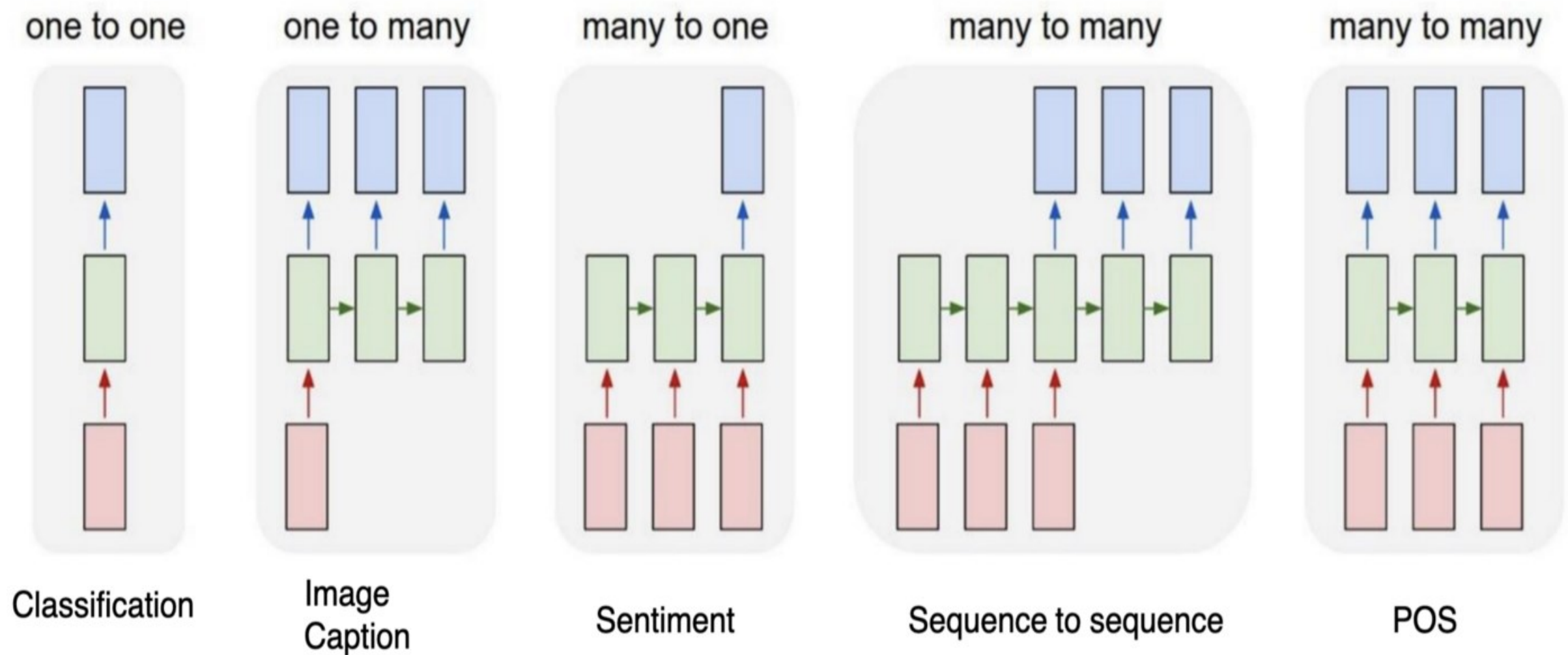
Recurrence is slow, improved by Mamba/RWKV (future class)

Recurrent Neural Networks (RNN)

A family of neural architectures



Types of RNN



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(Uh^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

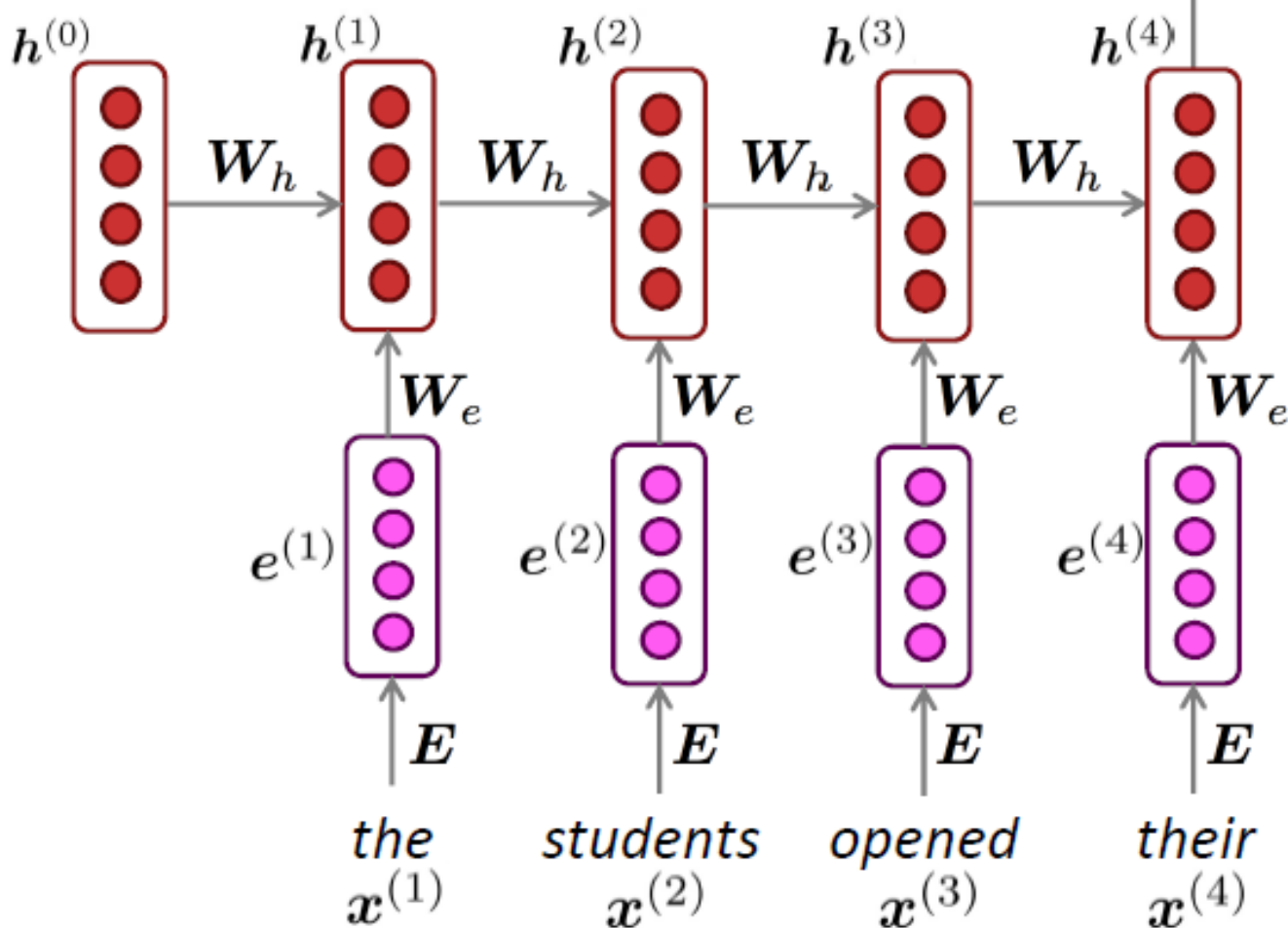
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = Ex^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$

Note: this input sequence could be much longer now!

Training an RNN Language Model

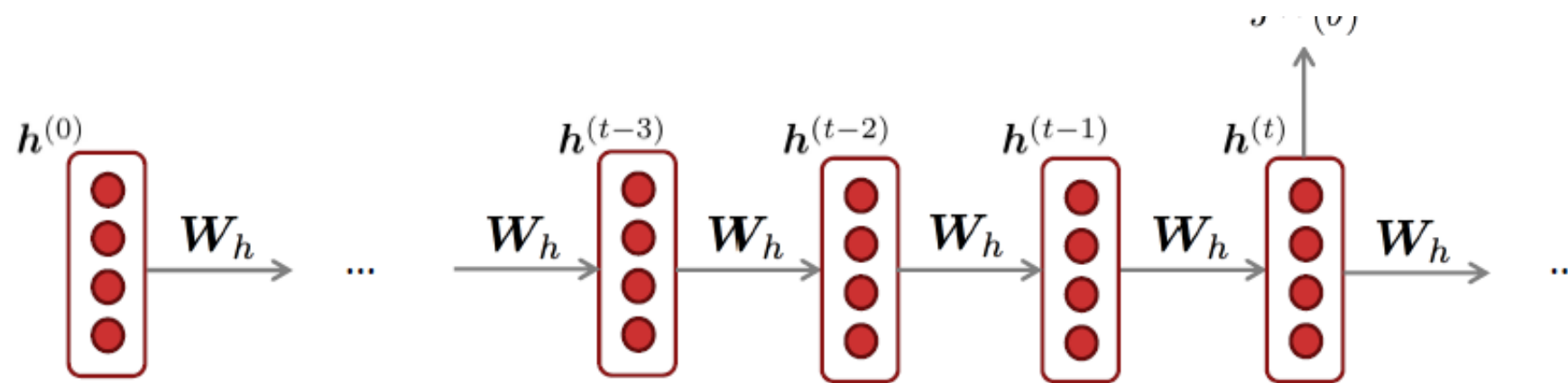
- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\mathbf{y}}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{\mathbf{y}}^{(t)}$, and the true next word $\mathbf{y}^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

Backpropagation for RNNs



Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the **repeated** weight matrix W_h ?

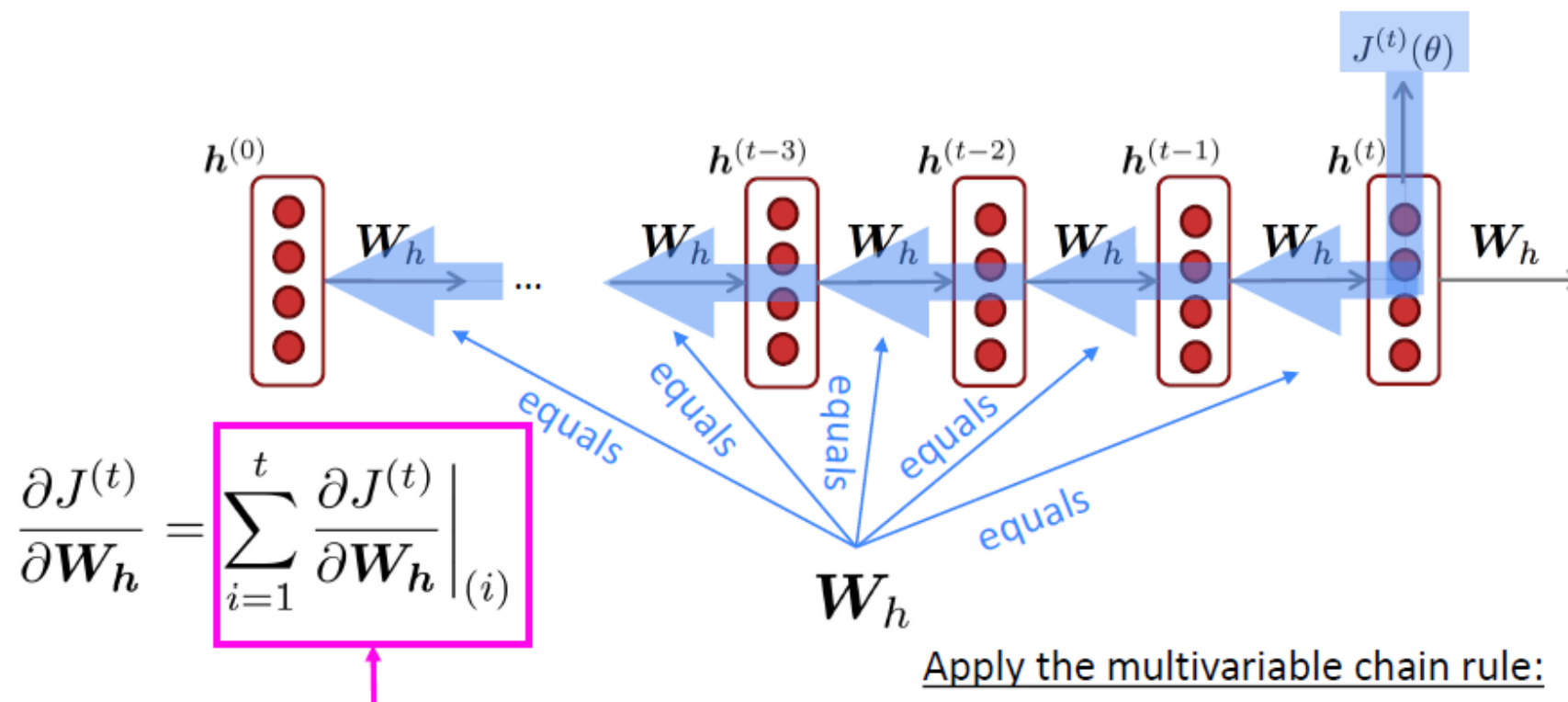
Answer:
$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \frac{\partial J^{(i)}}{\partial W_h} \Big|_{(i)}$$

“The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears”

Why?

Backpropagation for RNNs

In practice, often
“truncated” after
~20 timesteps for
training efficiency
reasons



Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \dots, 0$, summing gradients as you go. This algorithm is called “**backpropagation through time**” [Werbos, P.G., 1988, *Neural Networks 1*, and others]

Apply the multivariable chain rule:
= 1

$$\begin{aligned} \frac{\partial J^{(t)}}{\partial W_h} &= \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial W_h} \Big|_{(i)} \boxed{\frac{\partial W_h|_{(i)}}{\partial W_h}} \\ &= \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial W_h} \Big|_{(i)} \end{aligned}$$

Backpropagation through time

$$\begin{aligned}\mathbf{h}_1 &= g(\mathbf{W}\mathbf{h}_0 + \mathbf{U}\mathbf{x}_1 + \mathbf{b}) \\ \mathbf{h}_2 &= g(\mathbf{W}\mathbf{h}_1 + \mathbf{U}\mathbf{x}_2 + \mathbf{b}) \\ \mathbf{h}_3 &= g(\mathbf{W}\mathbf{h}_2 + \mathbf{U}\mathbf{x}_3 + \mathbf{b}) \quad \hat{\mathbf{y}}_3 = \text{softmax}(\mathbf{W}_o\mathbf{h}_3) \\ L_3 &= -\log \hat{\mathbf{y}}_3(w_4)\end{aligned}$$

First, compute gradient with respect to hidden vector of last time step: $\frac{\partial L_3}{\partial \mathbf{h}_3}$

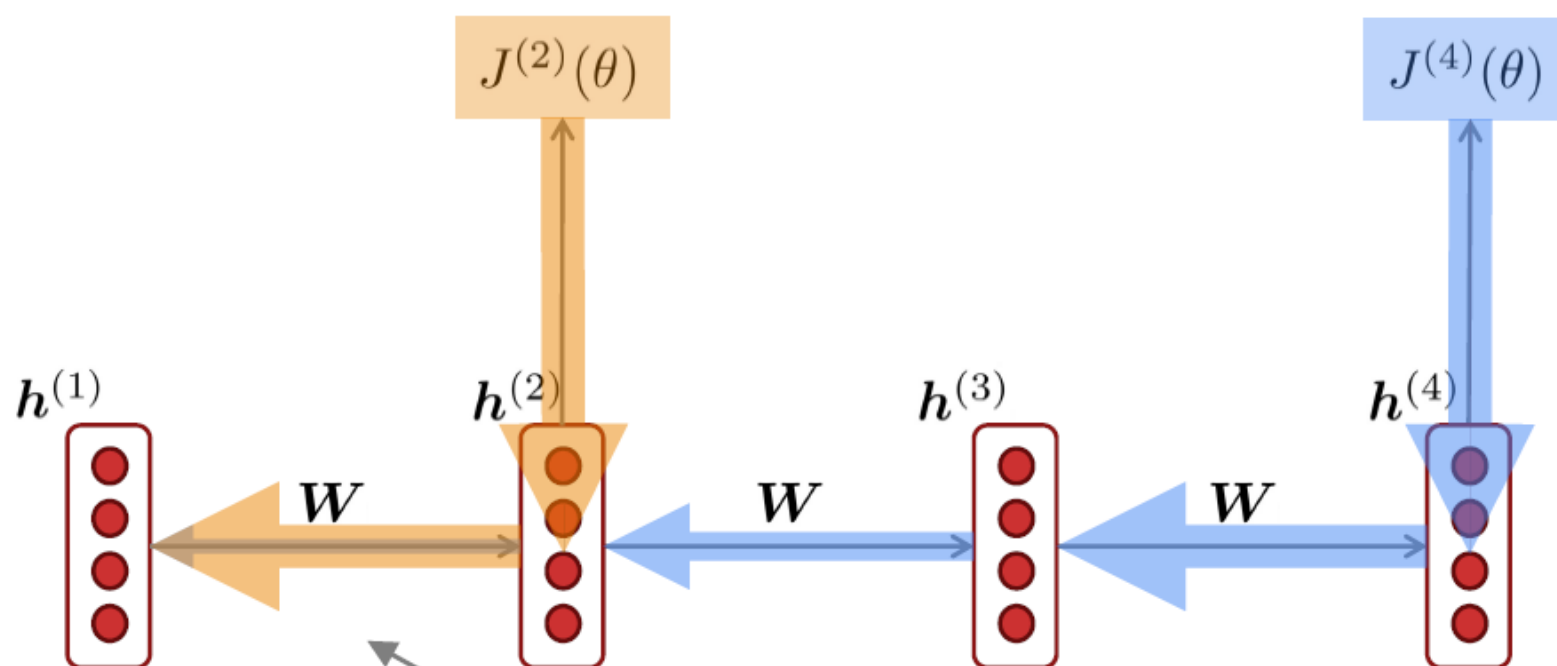
$$\frac{\partial L_3}{\partial \mathbf{W}} = \frac{\partial L_3}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{W}} + \frac{\partial L_3}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{W}} + \frac{\partial L_3}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{W}}$$

More generally,

$$\boxed{\frac{\partial L}{\partial \mathbf{W}} = -\frac{1}{n} \sum_{t=1}^n \sum_{k=1}^t \frac{\partial L_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}}$$

If k and t are far away, the gradients can grow/shrink exponentially (called **the gradient exploding or gradient vanishing problem**)

Why is vanishing gradient a problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are updated only with respect to near effects, not long-term effects.

The vanishing gradient problem for language models

Example (RNN-LM task):

Jane walked into the room. John walked in too. It was late in the day. Jane said hi to _____

To learn from this training example, the RNN-LM needs to **model the dependency** between “John” on the 7th step and the target word “John” at the end.

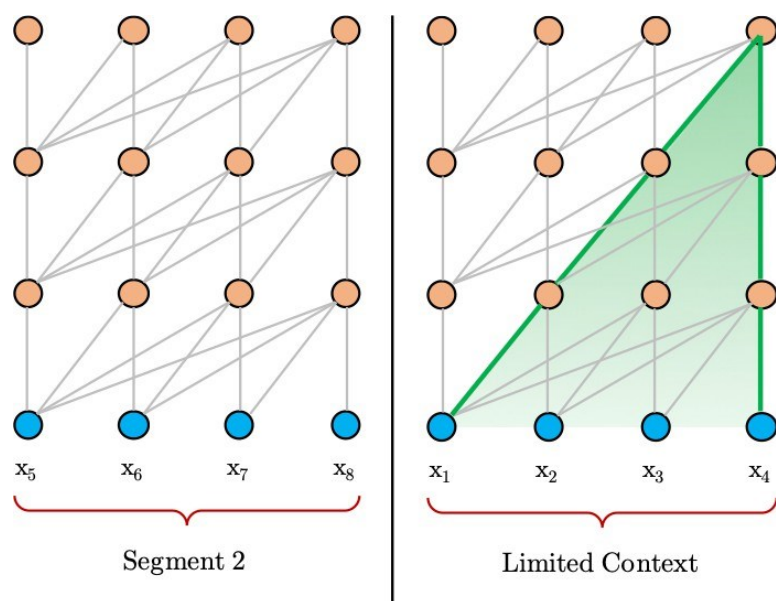
But if the gradient is small, the model **can't learn this dependency**

So, the model is **unable to predict similar long-distance dependencies** at test time

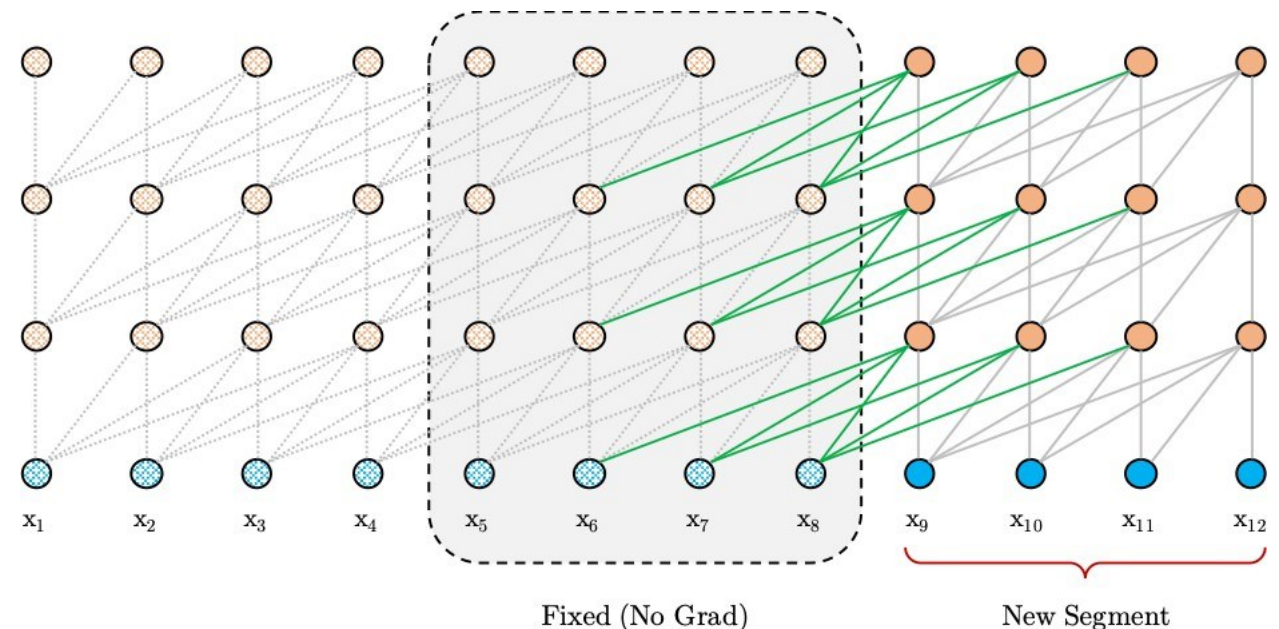
Truncated BPTT+Transformer

- Transformer-XL (Dai et al. 2019) attends to fixed **vectors** from the previous sentence

Standard Transformer



Transformer-XL

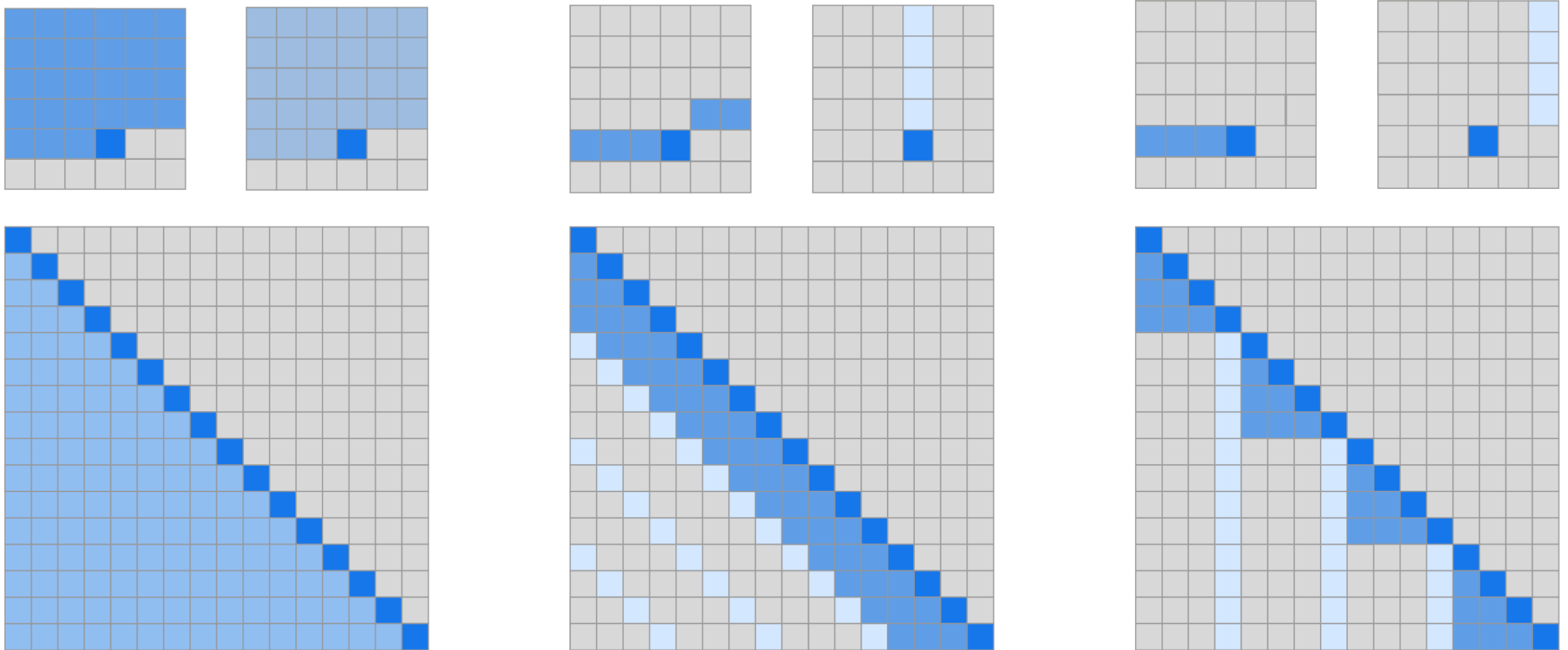


- Like truncated backprop through time for RNNs; can use previous states, but not backprop into them
- See also Mistral's (Jiang et al. 2023) sliding window attention

Sparse Transformers

(Child et al. 2019)

- Add "stride", only attending to every n previous states



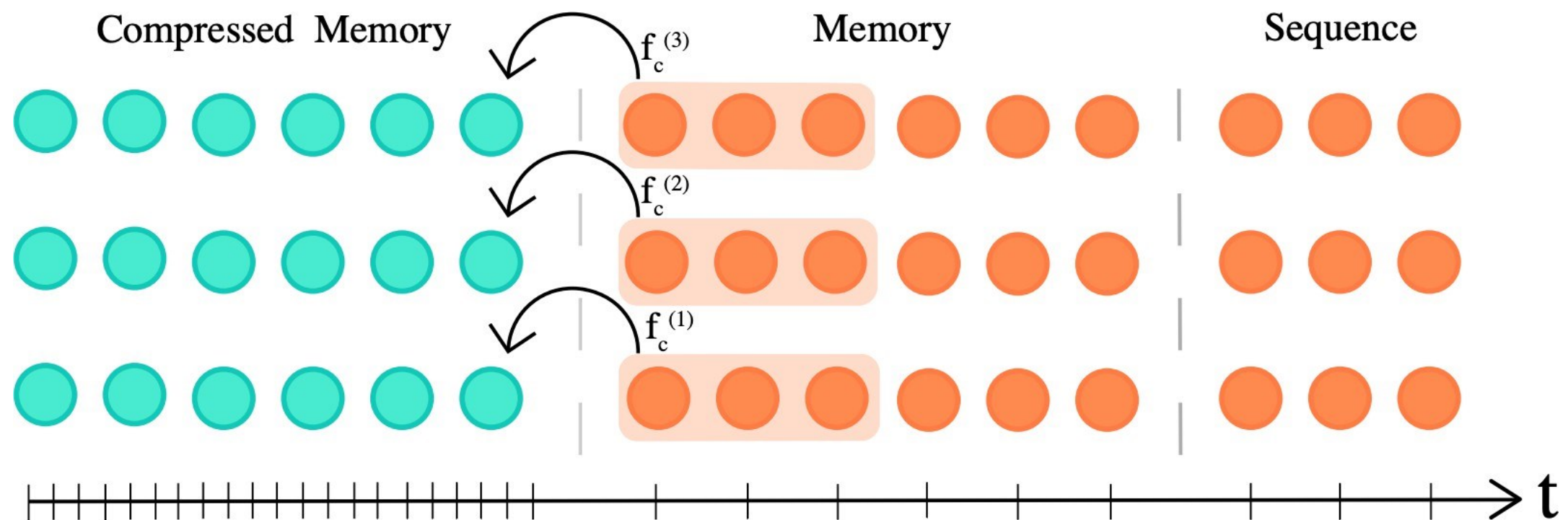
(a) Transformer

(b) Sparse Transformer (strided)

(c) Sparse Transformer (fixed)

Compressing Previous States

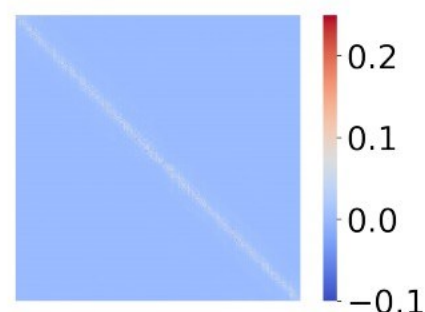
- Add a "strided" compression step over previous states (Rae et al. 2019)



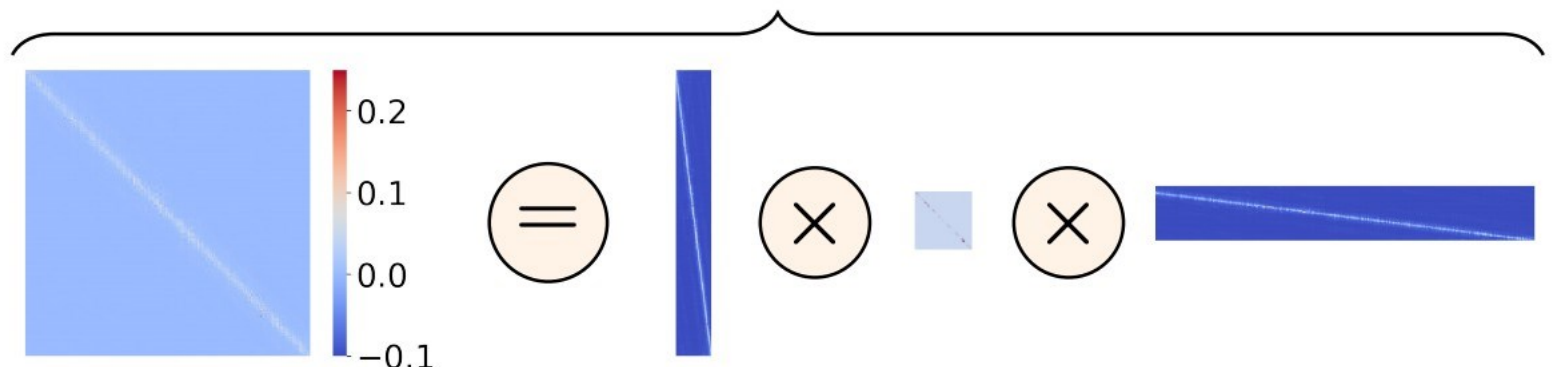
Low-rank Approximation

- Calculating the attention matrix is expensive, can it be predicted with a low-rank matrix?
- **Linformer:** Add low-rank linear projections into model (Wang et al. 2020)
- **Nystromformer:** Approximate using the Nystrom method, sampling "landmark" points (Xiong et al. 2021)

softmax

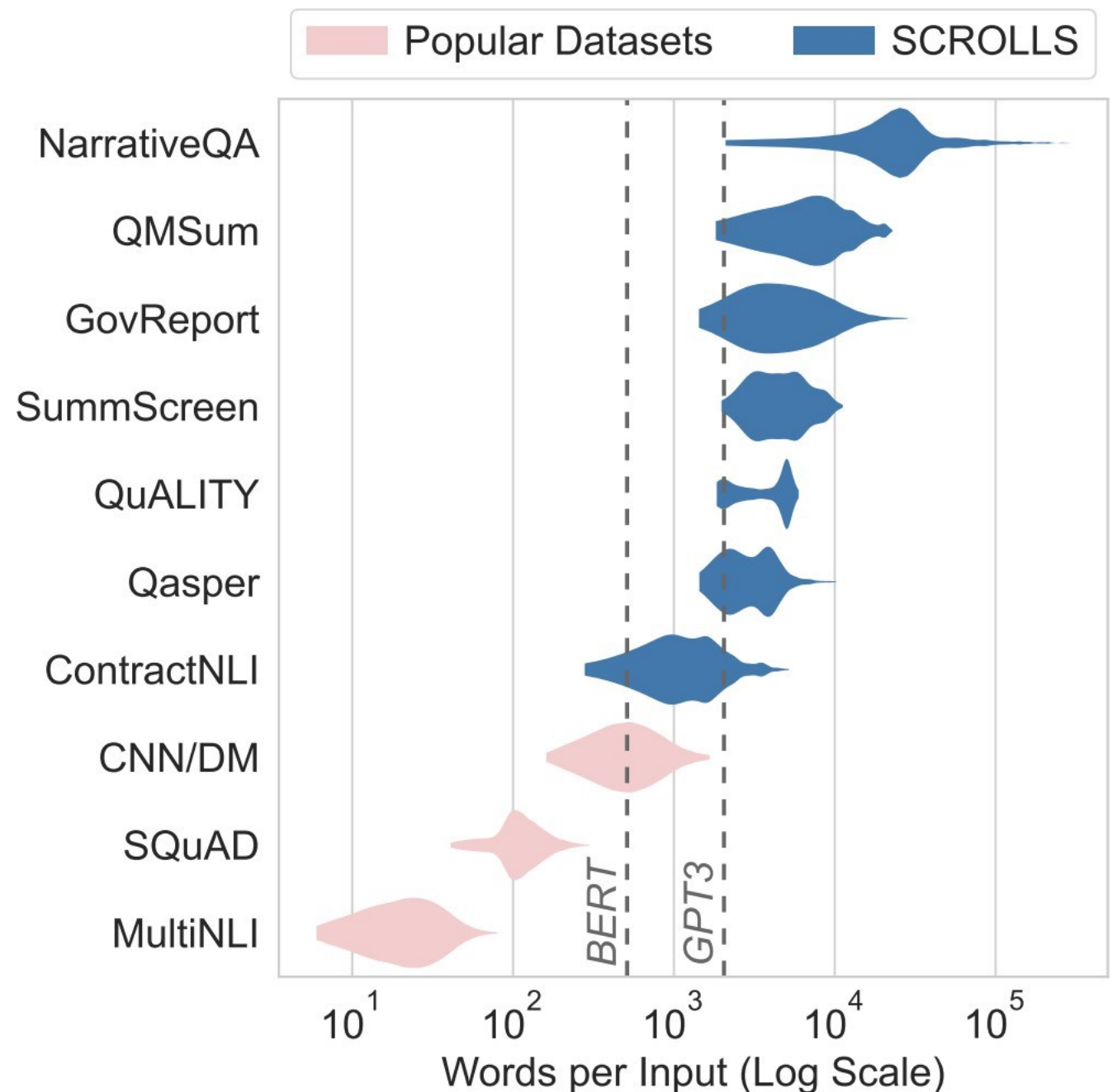


Nyström approximation



Benchmarks for Long-context Models

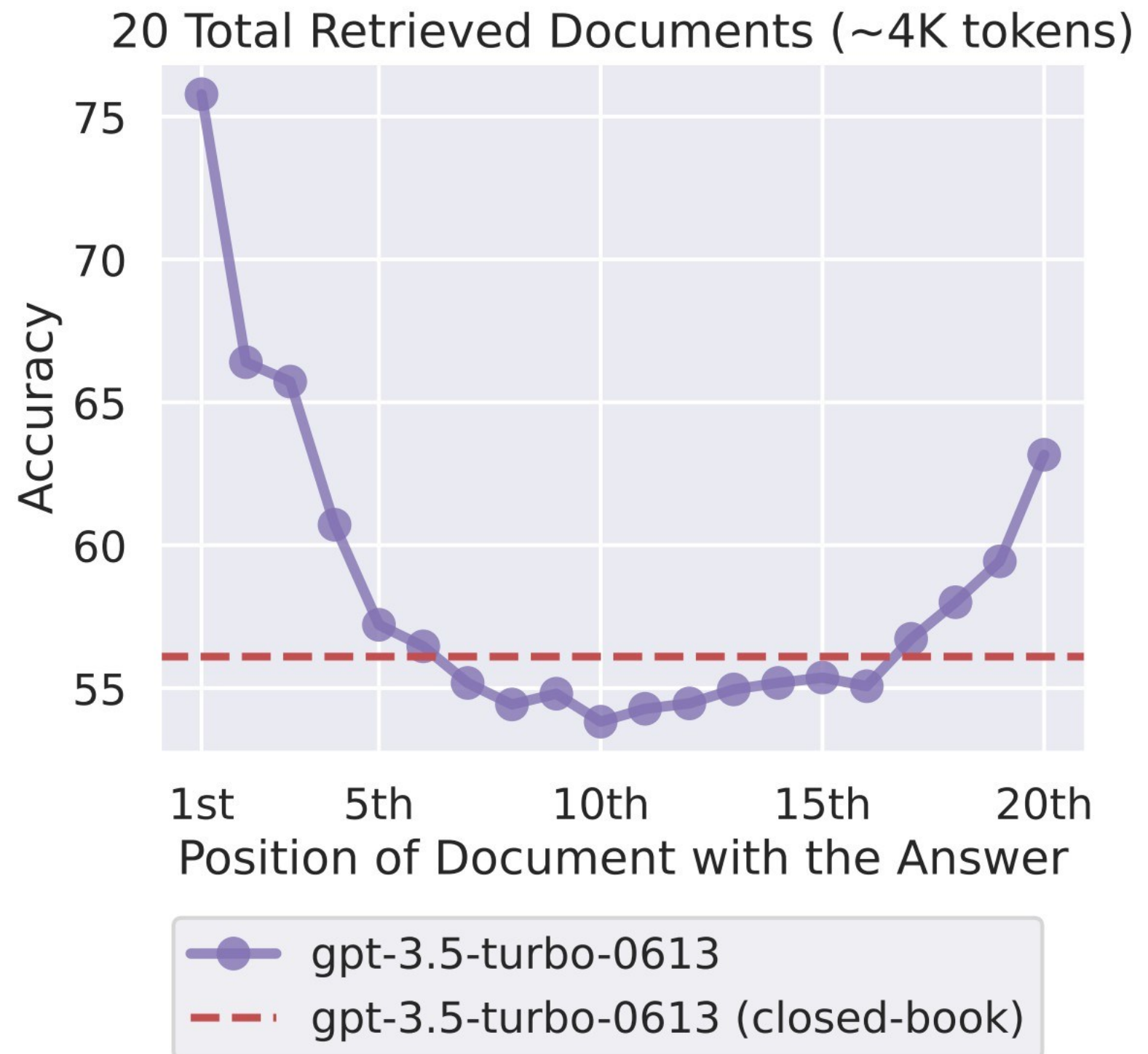
- **Long Range Arena:** Composite benchmark containing mostly non-NLP tasks (Tay et al. 2020)
- **SCROLLS:** Benchmark containing long-context summarization, QA, etc. (Shaham et al. 2022)



Effectively Using Long Contexts

As Context Increases, Models Miss Relevant Info

- e.g. “lost-in-the-middle” (Liu et al. 2023) demonstrates that models pay less attention to things in the middle of context windows



Ensuring Use of Relevant Context

- Better retrievers make more relevant context
- Decide whether to include passages (Asai et al. 2021)
- Filter down to parts of retrieved passages (Wang et al. 2023)

Retrieved Passage

infrastructure necessary for rapid industrial growth was put in place. The first railway in Belgium, running from northern Brussels to Mechelen, was completed in May 1835. The earliest railway in Britain was a wagonway system, a horse drawn wooden rail system, used by German miners at Caldbeck, Cumbria, England, perhaps from the 1560s. A wagonway was built at Prescott, near Liverpool, sometime around 1600, possibly as early as 1594. Owned by Philip Layton, the line carried coal from a pit near Prescott Hall to a terminus about half a mile away. On 26 July 1803, Jessop opened the Surrey Iron

Question 

When did the first train run in England?

Generator

1835 

1560s 

Distilled Content

The earliest railway in Britain was a wagonway system, a horse drawn wooden rail system, used by German miners at Caldbeck, Cumbria, England, perhaps from the 1560s.

Resources

<https://phontron.com/class/anlp2024/lectures/#retrieval-and-rag-feb-15>

Recommended Reading: [ACL 2023 RAG Tutorial](#) (Asai et al. 2023)

Reference: [Retrieval-based QA](#) (Chen et al. 2017)

Reference: [Dense Passage Retrieval](#) (Karpukhin et al. 2020)

Reference: [Introduction to Information Retrieval](#) (Manning et al. 2009)

Software: [Apache Lucene](#)

Reference: [DPR](#) (Karpukhin et al. 2020)

Reference: [Contriever](#) (Izacard et al. 2022)

Software: [FAISS](#)

Software: [ChromaDB](#)

Reference: [Cross-encoder Reranking](#) (Nogueira et al. 2019)

Reference: [Token-level Retrieval](#) (Khattab and Zaharia 2020)

Reference: [Hypothetical Document Embeddings](#) (Gao et al. 2022)

Reference: [End-to-end RAG Training](#) (Lewis et al. 2020)

Reference: [Toolformer](#) (Schick et al. 2023)

Reference: [FLARE](#) (Jiang et al. 2023)

Reference: [kNN-LM](#) (Khandelwal et al. 2019)

Reference: [Unlimiformer](#) (Bertsch et al. 2023)

Reference: [Training Transformers with Context](#) (Voita et al. 2018)

Reference: [Transformer-XL](#) (Dai et al. 2019)

Reference: [Mistral](#) (Jiang et al. 2023)

Reference: [Sparse Transformers](#) (Child et al. 2019)

Reference: [Compressive Transformer](#) (Rae et al. 2019)

Reference: [Liformer](#) (Wang et al. 2020)

Reference: [Nystromformer](#) (Xiong et al. 2021)

Reference: [Long Range Arena](#) (Tay et al. 2020)

Reference: [SCROLLS](#) (Shaham et al. 2022)

Reference: [Lost in the Middle](#) (Liu et al. 2023)

Reference: [Deciding Whether to Use Passages](#) (Asai et al. 2021)

Reference: [Learning to Filter Context](#) (Wang et al. 2023)